



SIMD

2025

André Pereira

Introduction



- SIMD architectures can exploit significant data-level parallelism for:
 - matrix-oriented scientific computing
 - media-oriented image and sound processing
- SIMD is more energy efficient than MIMD
 - only needs to fetch one instruction per data operation
 - makes SIMD attractive for personal mobile devices
- SIMD allows programmers to continue to think sequentially

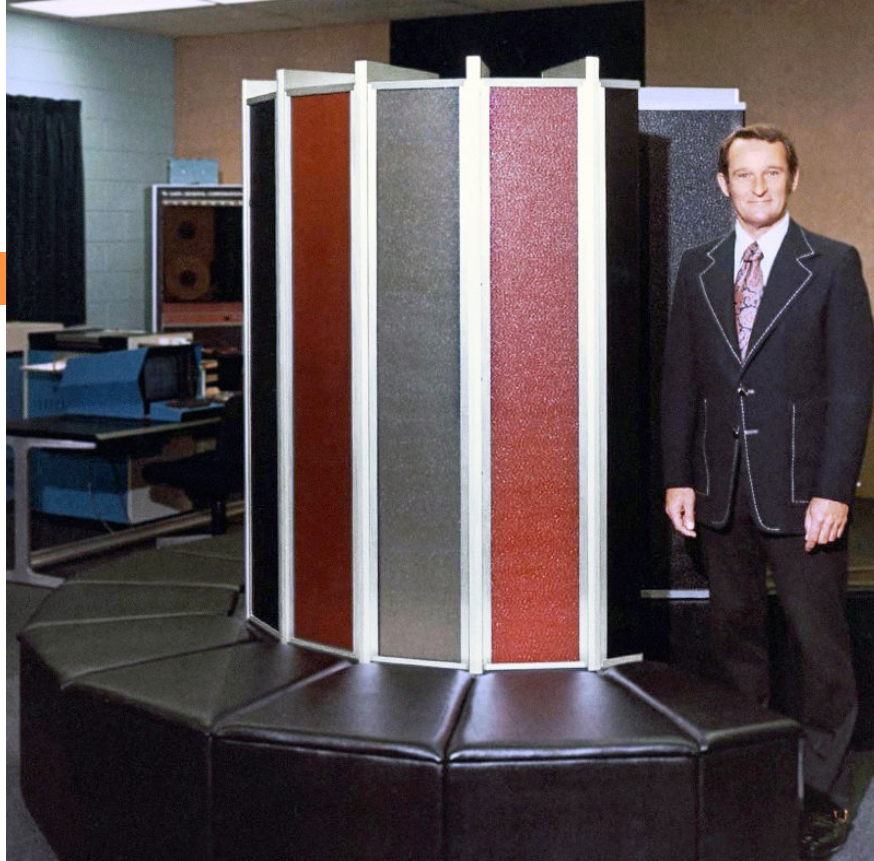
SIMD Parallelism



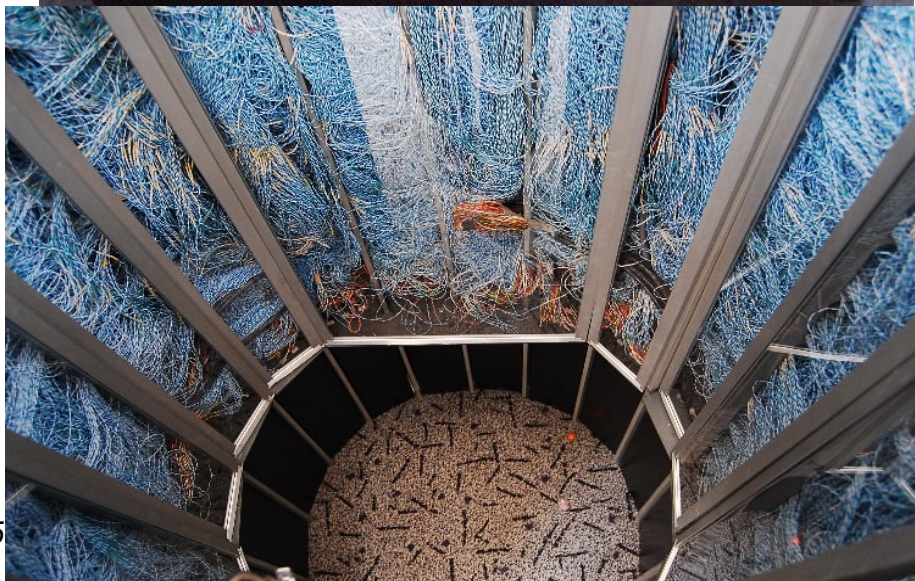
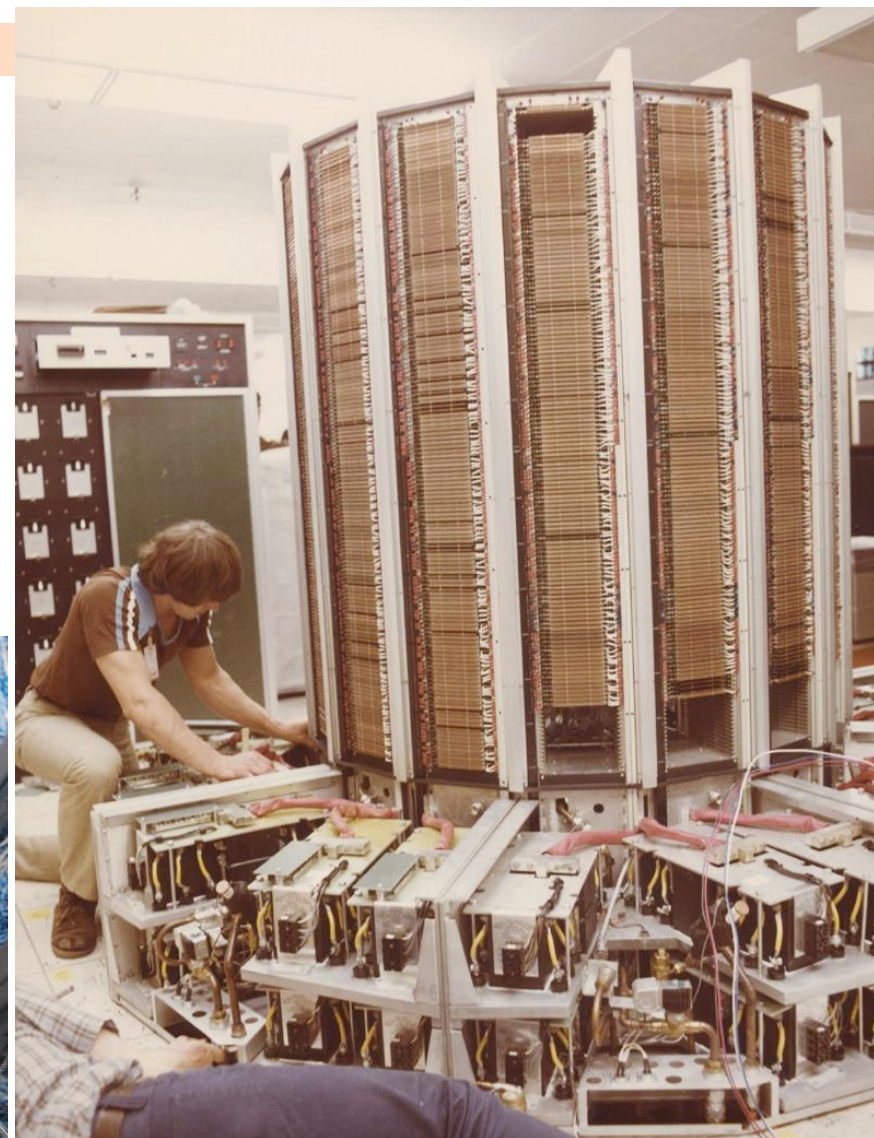
- Vector architectures
 - Read sets of data elements (*gather from memory*) into “vector registers”
 - Operate on those registers
 - Store/*scatter* the results back into memory
- SIMD & extensions on scalar processors
- Graphics Processor Units (GPUs)
(*next set of slides*)



- Basic idea:
 - Read sets of data elements (*gather from memory*) into “vector registers”
 - Operate on those registers
 - Store/*scatter* the results back into memory
- Registers are controlled by the compiler
 - Used to hide memory latency
 - Leverage memory bandwidth



Cray-1 Supercomputer (1976)

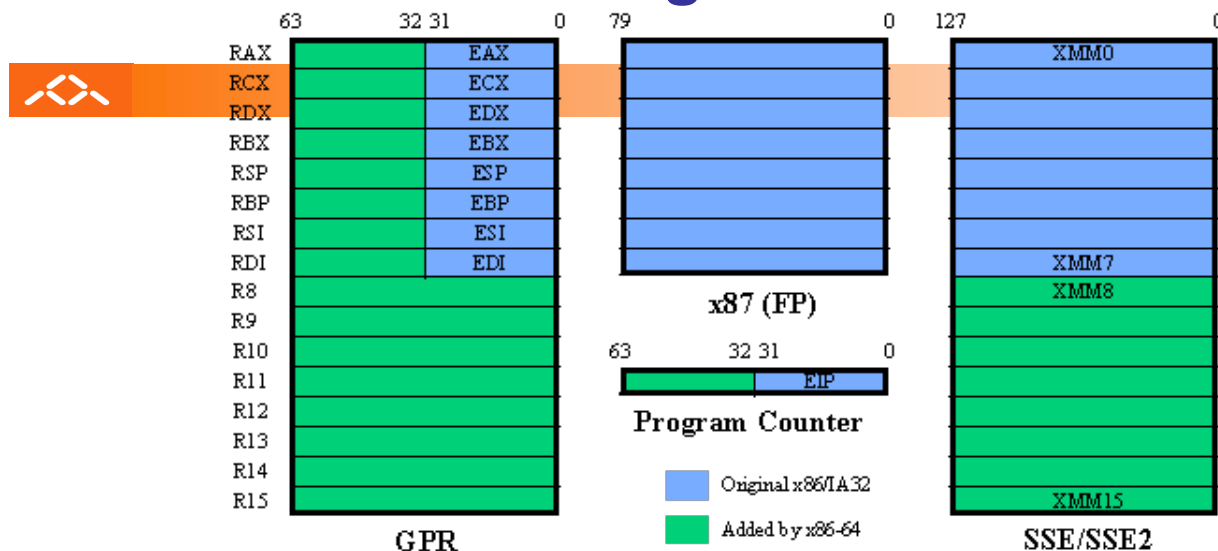




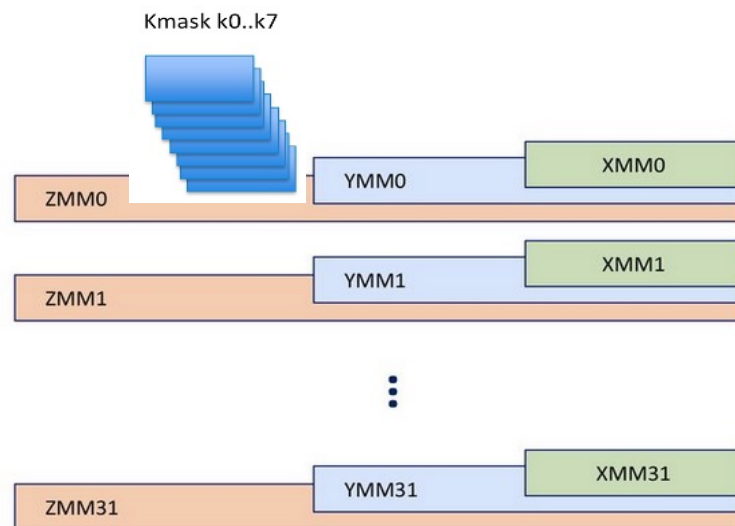
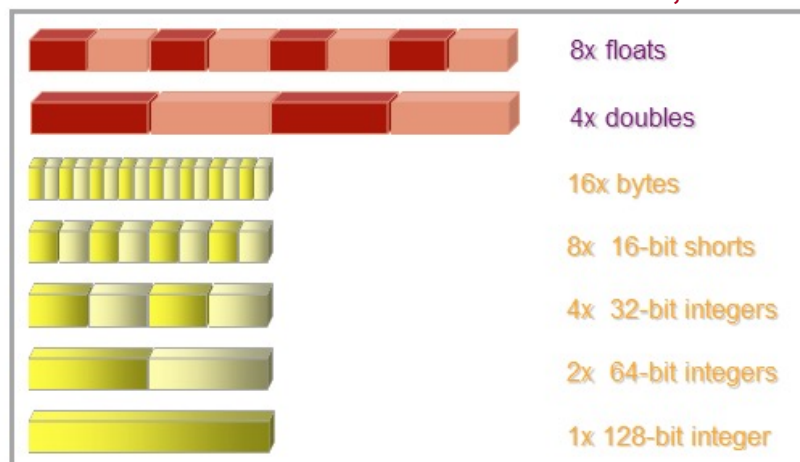
- Compilers are a key element to give hints on whether a code section will vectorize or not
- Check if loop iterations have data dependencies and/or `if...then` statements, otherwise vectorization is compromised
- Vector architectures have a too high cost, but simpler variants are currently available on off-the-shelf devices, as extensions to the scalar processor; **however:**
 - most do not support **non-unit stride** => care must be taken in the design of data structures
 - same applies for **mask register, gather-scatter...**

- Intel implementations:
 - MMX (1996)
 - Eight 8-bit integer ops or four 16-bit integer ops
 - Streaming SIMD Extensions (SSE) (1999)
 - Eight 16-bit integer ops
 - Four 32-bit integer/fp ops or two 64-bit integer/fp ops
 - Advanced Vector eXtensions (AVX) (2010...)
 - Eight 32-bit fp or four 64-bit fp ops (integers only in AVX-2)
 - 512-bits wide in AVX-512
 - Operands must / should be in consecutive and aligned memory locations
- AMD Zen/Epyc (Opteron follow-up): up to AVX-512...
- Armv8 (64-bit) architecture: NEON & SVE

Registers for vector processing in Intel 64



Advanced Vector eXtensions, AVX 1.0



- ☐ XMM0 – XMM15
 - 128-bit registers
 - SSE
- ☐ YMM0 – YMM15
 - 256-bit registers
 - AVX, AVX2
- ☐ ZMM0 – ZMM31
 - 512-bit registers
 - AVX-512

AVX 2.0

AMD Zen 4 supports AVX-512, but...

The AVX-512 across Intel devices



Microarchitecture	Support Level													
	F	CD	ER	PF	BW	DQ	VL	IFMA	VBMI	4FMAPS	VNNI	4VNNIW	VPOPCNTDQ	BF16
Knights Landing	✓	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Knights Mill	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✓	✓	✓	✗
Skylake (server)	✓	✓	✗	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗
Cascade Lake	✓	✓	✗	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗	✗
Cannon Lake	✓	✓	✗	✗	✓	✓	✓	✓	✓	✗		✗	✗	✗
Ice Lake (server)	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗
Cooper Lake	✓	✓	✗	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗	✓

AVX512F - [AVX-512 Foundation](#)

AVX512CD - [AVX-512 Conflict Detection](#)

AVX512BW - [AVX-512 Byte and Word](#)

AVX512DQ - [AVX-512 Doubleword and Quadword](#)

AVX512VL - [AVX-512 Vector Length](#)

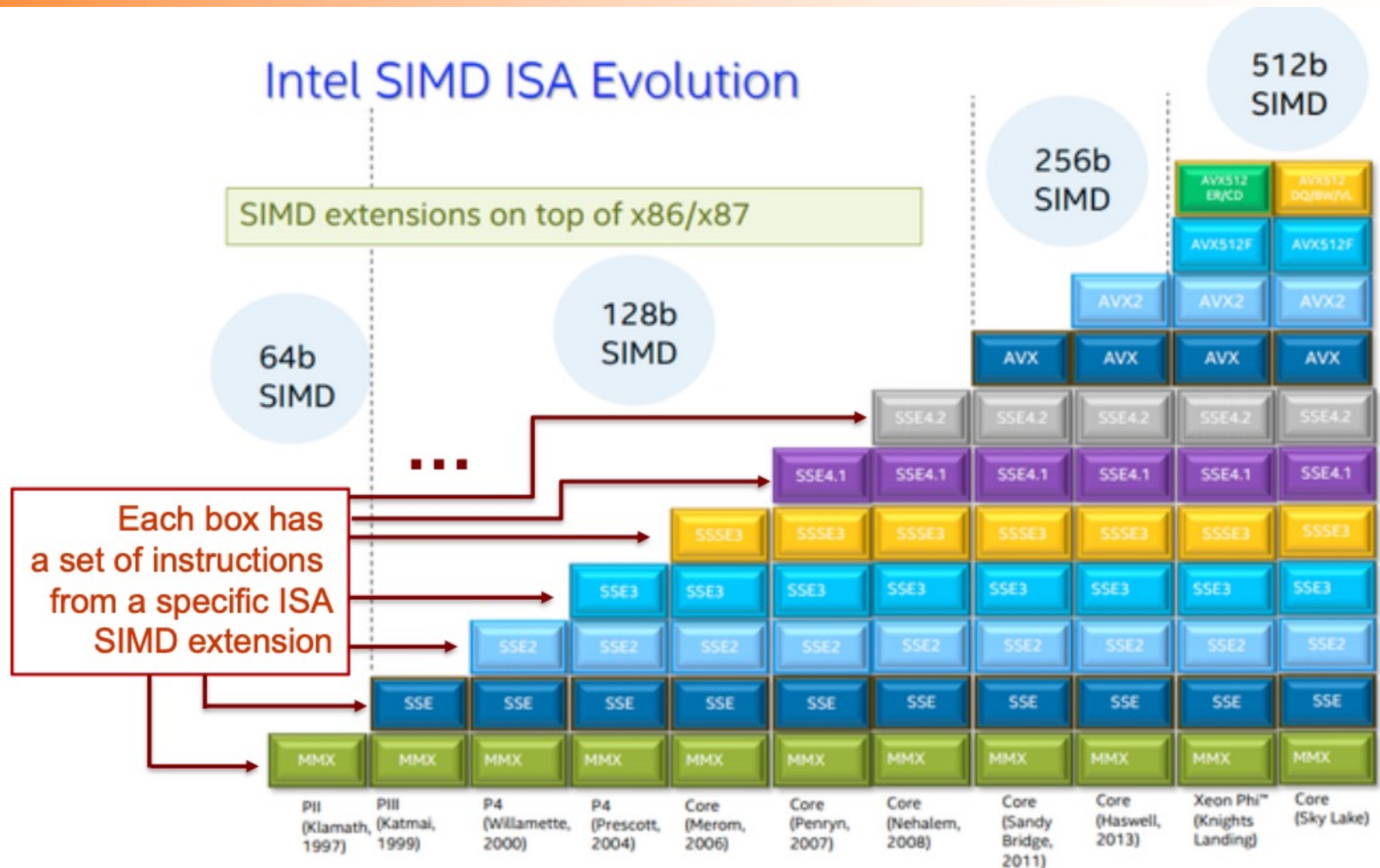
AVX512IFMA - [AVX-512 Integer Fused Multiply-Add](#)

AVX512_VNNI - [AVX-512 Vector Neural Network Instructions](#)

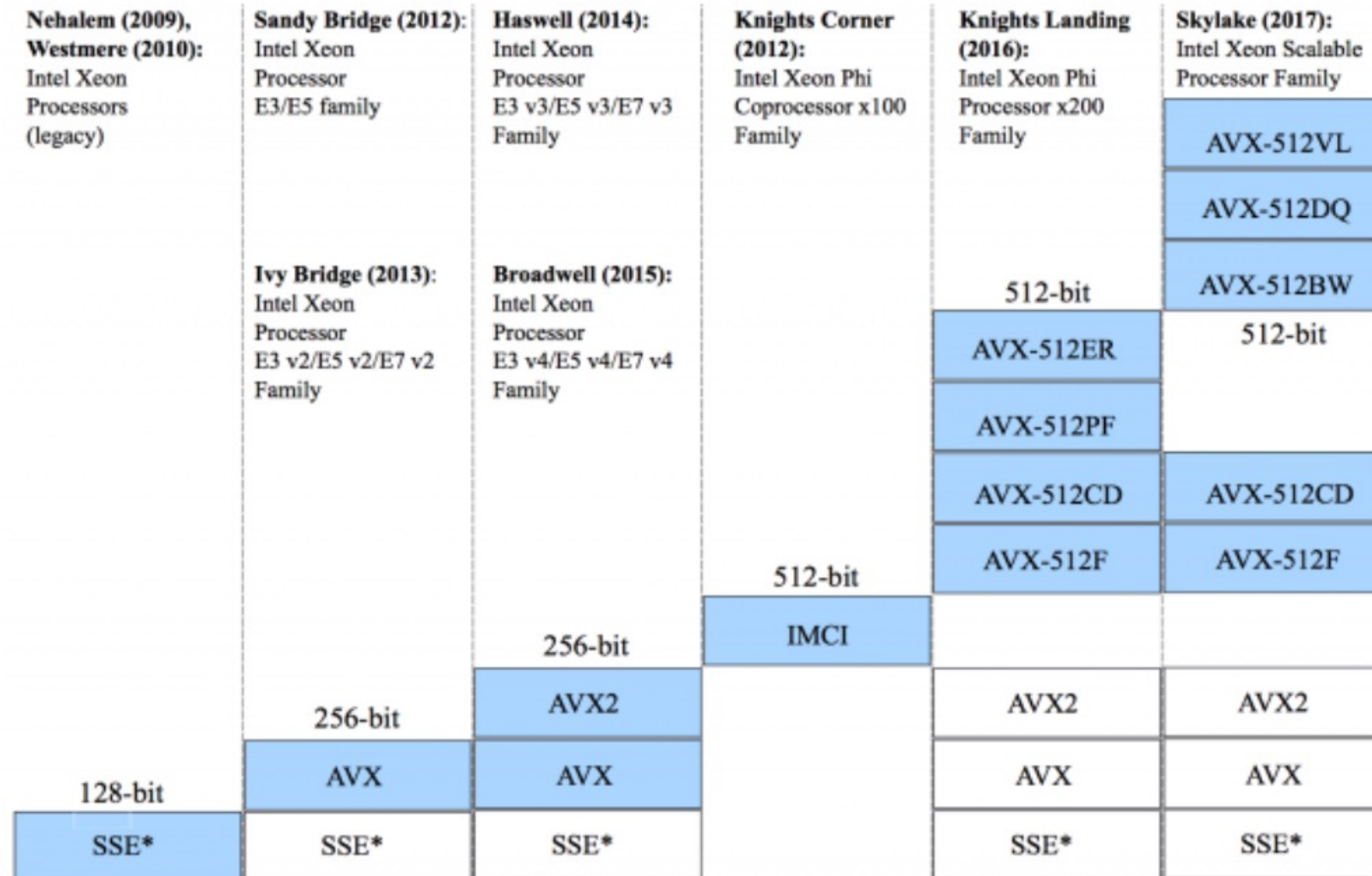
AVX512_BF16 - [AVX-512 BFloat16 Instructions](#)

*"I hope AVX512 dies a painful death,
and that Intel starts
fixing real problems..."*
Linus Torvalds

Intel SIMD Evolution



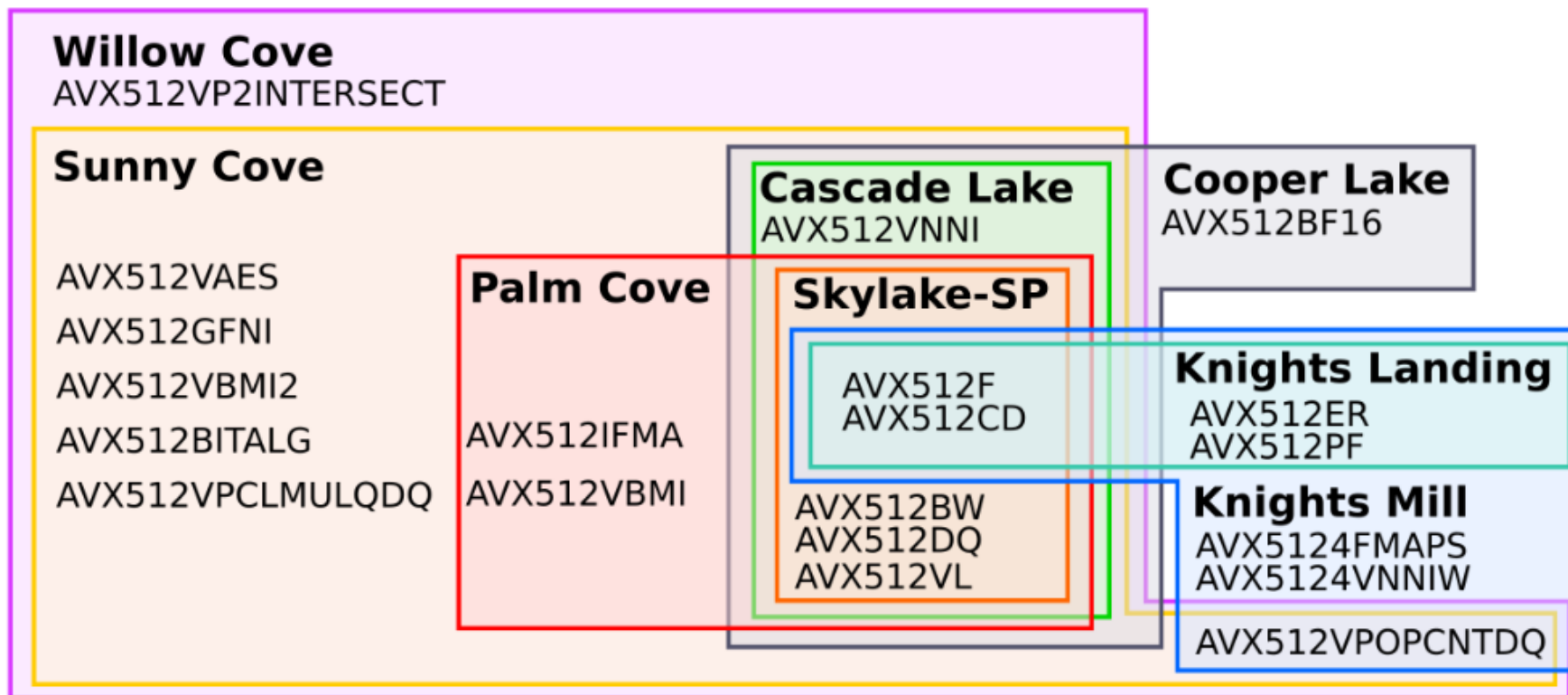
Intel SIMD Evolution



— primary instruction set

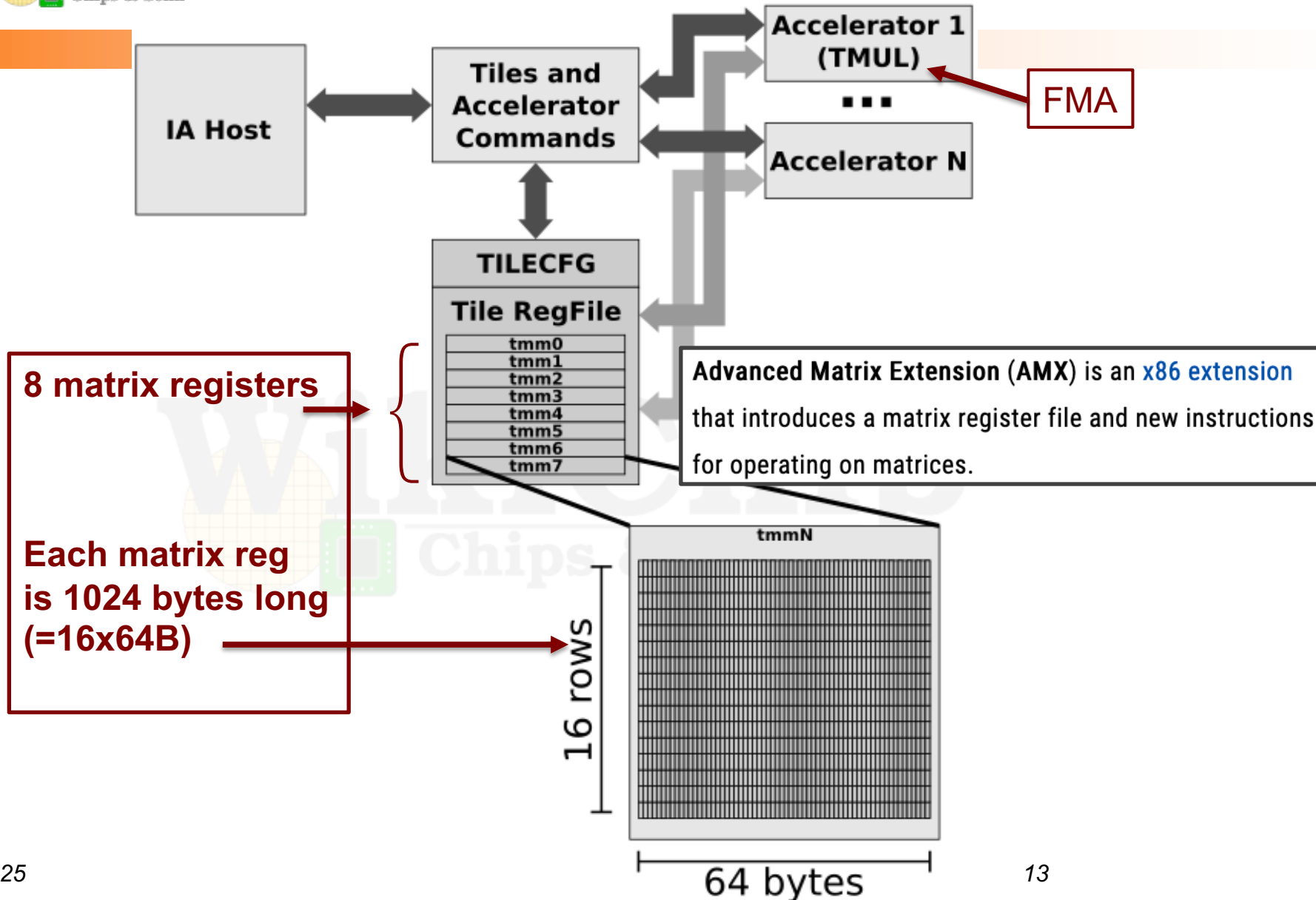
— legacy instruction set

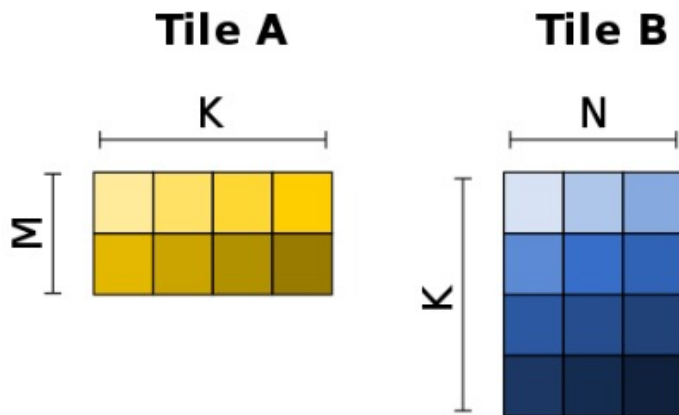
The AVX-512 Mess



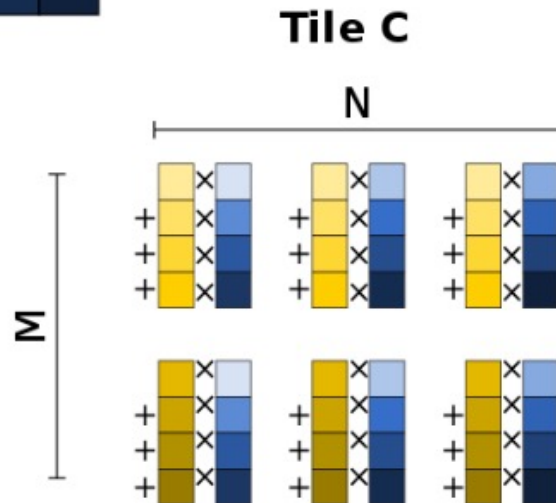
Intel AVX-512 Instruction Types	
AVX-512-F	AVX-512 Foundation Instructions
AVX-512-VL	Vector Length Orthogonality : ability to operate on sub-512 vector sizes
AVX-512-BW	512-bit Byte/Word support
AVX-512-DQ	Additional D/Q/SP/DP instructions (converts, transcendental support, etc.)
AVX-512-CD	Conflict Detect : used in vectorizing loops with potential address conflicts

Intel Advanced Matrix Extension (AMX) (expected in 2023)



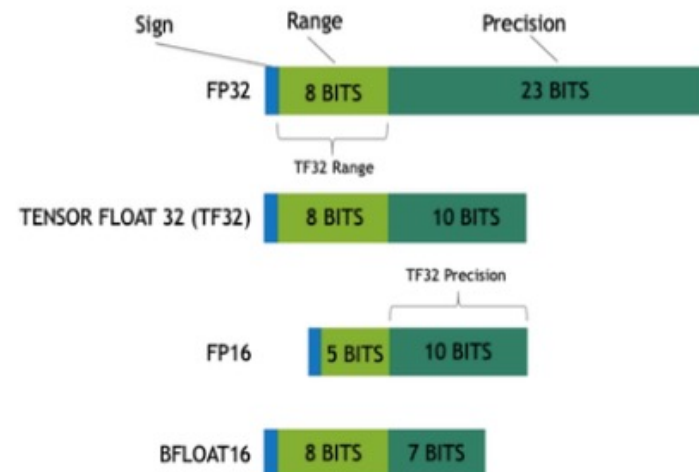


TMUL
 $C += A * B$



AMX Extensions		
Feature Set	Description	Instructions
AMX-TILE	The base matrix tile architecture support.	7 instructions
AMX-INT8	Dot-product of Int8 tiles.	4 instructions
AMX-BF16	Dot-product of BF16 tiles.	1 instruction

Data types



NEON vector & FP registers in Armv8 (64-bit)

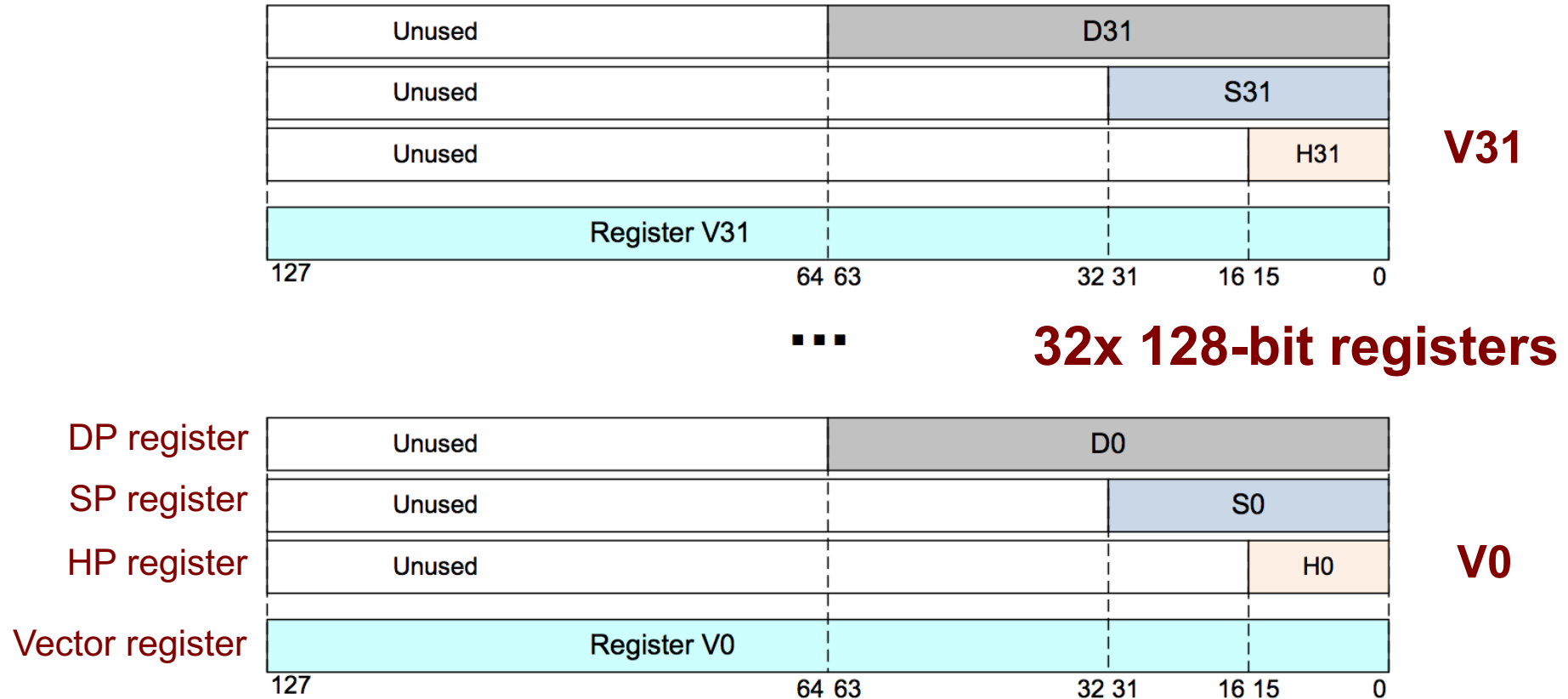


Figure 4-10 Arrangement of floating-point values

Note

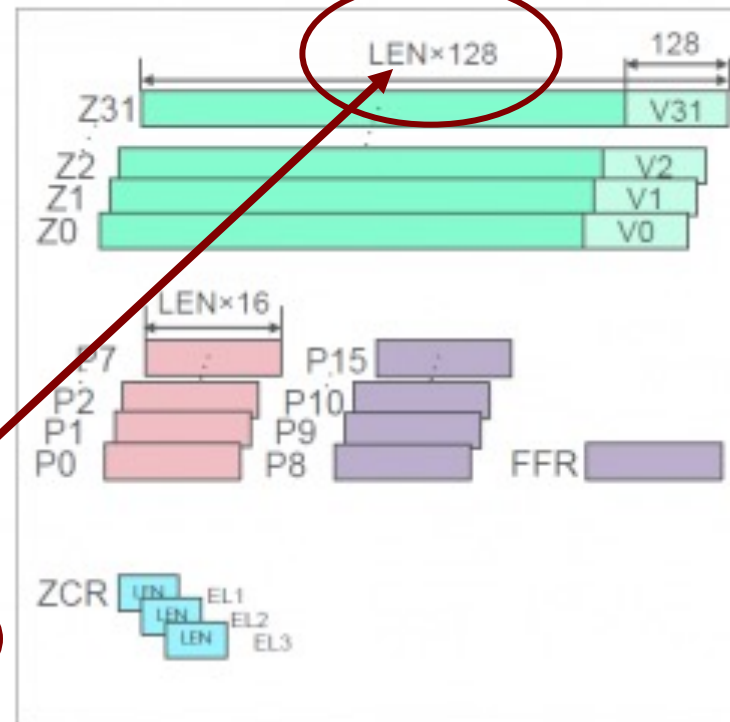
16-bit floating-point is supported, but only as a format to be converted from or to. It is not supported for data processing operations.

Armv8-A Scalable Vector Extension (SVE)



SVE architectural state

- Scalable vector registers
 - Z0-Z31 extending NEON's V0-V31
 - DP & SP floating-point
 - 64, 32, 16 & 8-bit integer
- Scalable predicate registers
 - P0-P7 lane masks for ld/st/arith
 - P8-P15 for predicate manipulation
 - FFR *first fault register*
- Scalable vector control registers
 - ZCR_ELx vector length (LEN=1..16)
 - Exception / privilege level EL1 to EL3



Vector size: up to **2048**

Beyond vector extensions



- Vector/SIMD-extended architectures are hybrid approaches
 - mix (super)**scalar + vector** op capabilities on a single device
 - **highly pipelined** approach to reduce memory access penalty
 - **tightly-closed access to shared memory**: lower latency
- Evolution of vector/SIMD-extended architectures
 - **computing accelerators optimized for number crunching (GPU)**
 - **add support for matrix multiply + accumulate operations; why?**
 - most scientific, engineering, AI & finance applications use matrix computations, namely the dot product: multiply and accumulate the elements in a row of a matrix by the elements in a column from another matrix
 - manufacturers typically call these extension **Tensor Processing Unit (TPU)**
 - **support for half-precision FP & 8-bit integer; why?**
 - machine learning using neural nets is becoming very popular; to compute the model parameter during training phase, intensive matrix products are used and with very low precision (is adequate!)